Markdown Studio — Feature Demo

Open this file and run **Markdown Studio: Preview** (`Cmd+Shift+P`).

- Markdown Studio — Feature Demo
  - 1. Markdown Rendering
  - 2. Mermaid Diagrams
    - Markdown Studio Architecture
    - Extension Activation Flow
  - 3. PlantUML Diagrams
    - Extension Component Diagram
    - Document Processing Sequence
  - 4. Inline SVG (Sanitized)
  - 5. Syntax Highlighting
  - 6. Images and Security
    - ✅ Local image (relative path — works)
    - ✅ Local image (absolute path — works)
    - ❌ External image (blocked by policy)
    - ❌ External link (blocked by policy)
    - How to allow external resources
    - Security Summary
  - 7. Theme Adaptability
  - 8. Diagram Type Catalog
  - 9. Custom CSS
  - 10. PDF Export

---

1. Markdown Rendering

Bold, *italic*, ~~strikethrough~~, `inline code`

> Blockquotes work too.

Feature	Status
Markdown	✅
Mermaid	✅
PlantUML	✅
SVG	✅
PDF Export	✅

1. Ordered list item
2. Another item
3. Third item

- Unordered item
- Another item

- Item

- 日本語
  - 2byte文字テスト

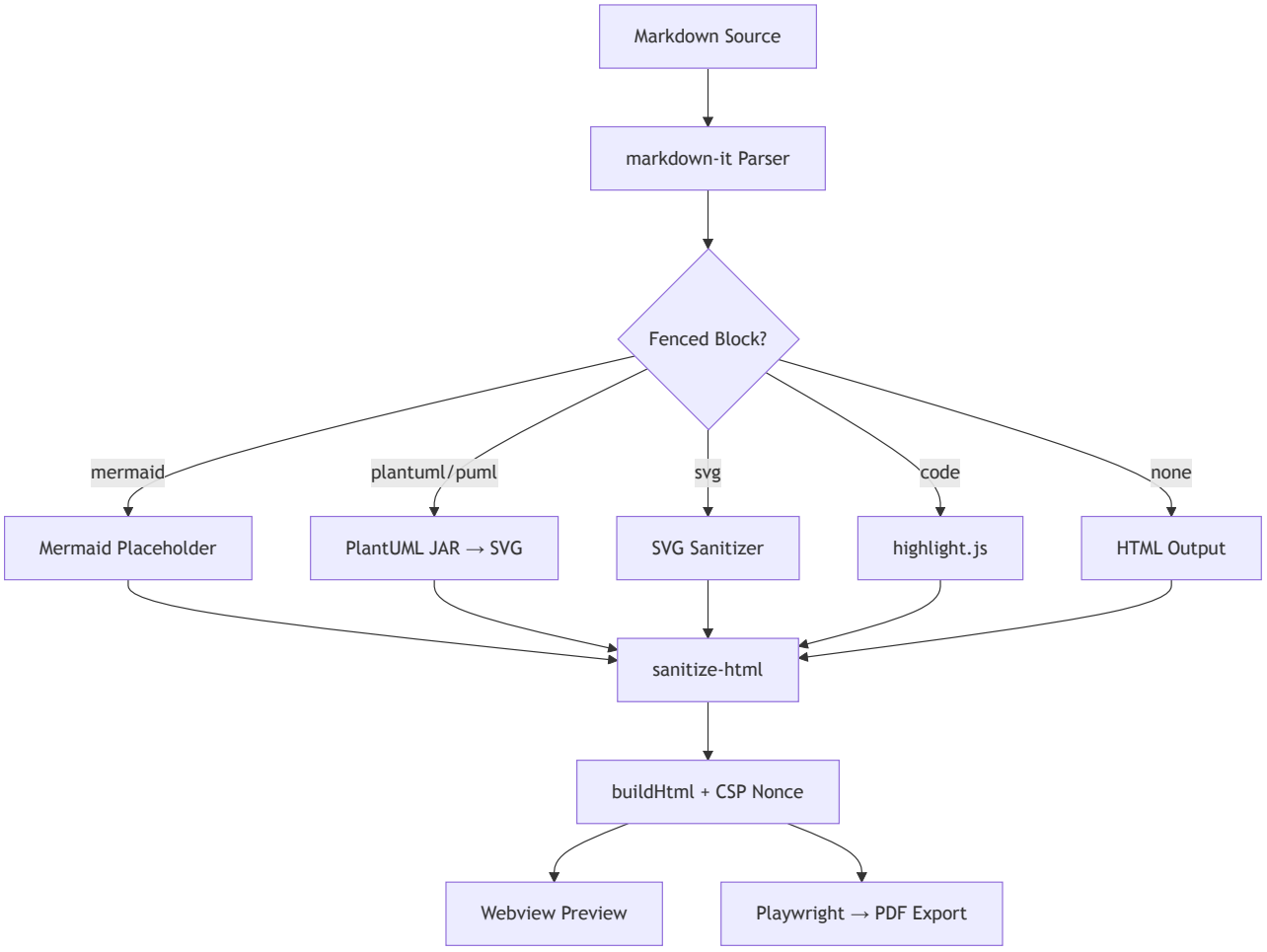
Task Lists

- [x] Markdown rendering
- [x] Mermaid diagrams
- [x] PlantUML diagrams
- [ ] LaTeX math (planned)
- [ ] Footnotes (planned)

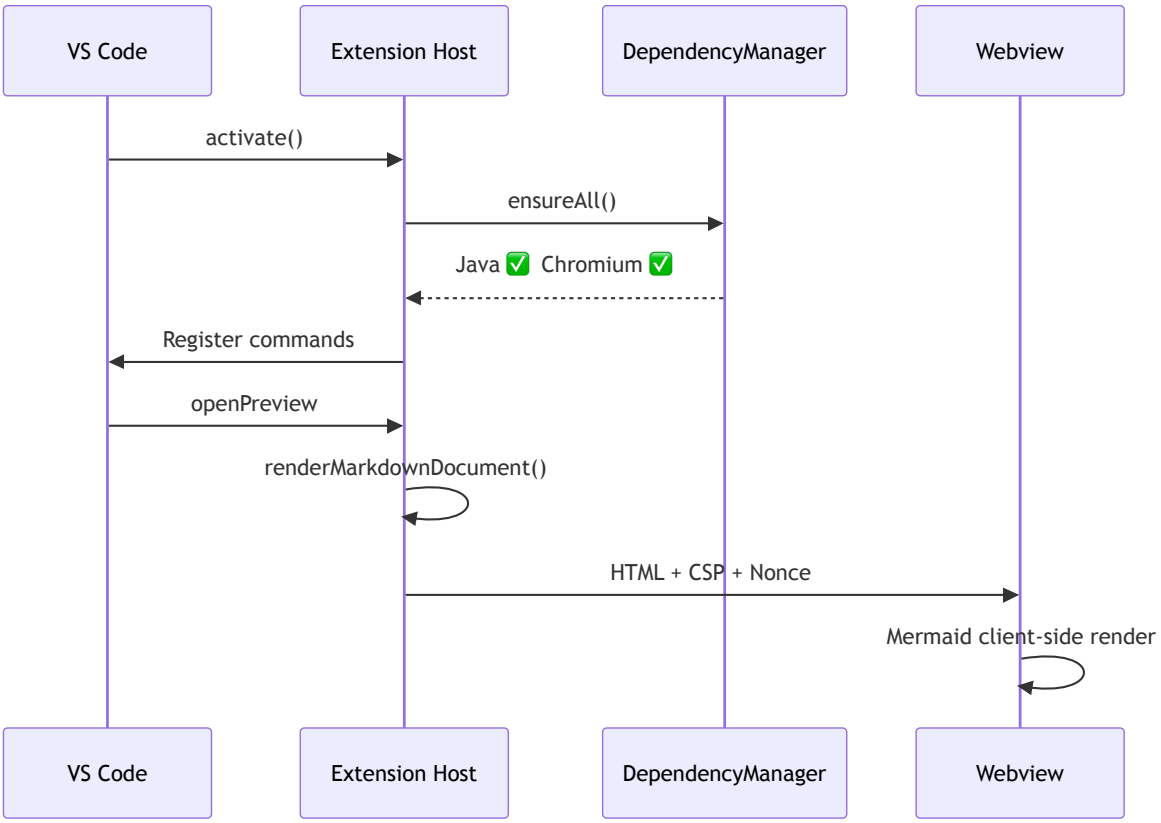
---

2. Mermaid Diagrams

Markdown Studio Architecture



Extension Activation Flow

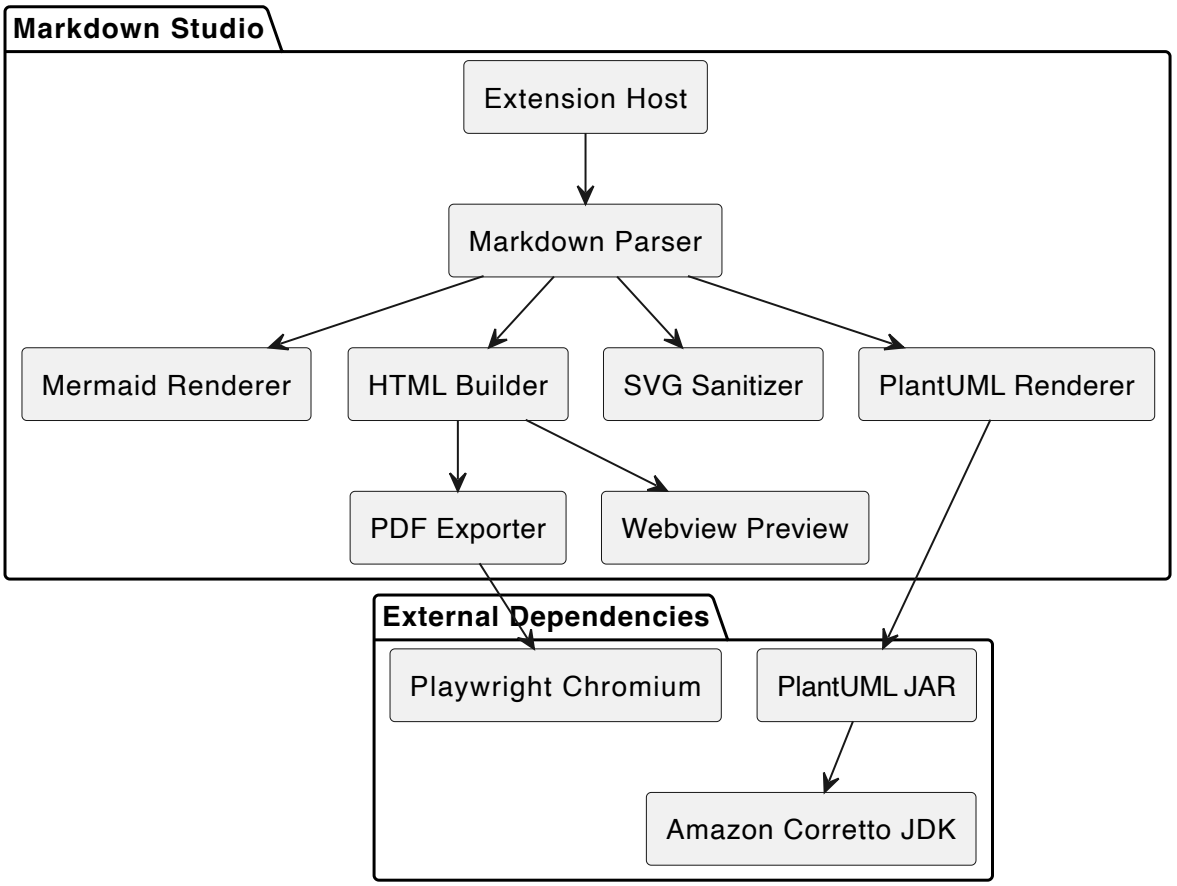


---

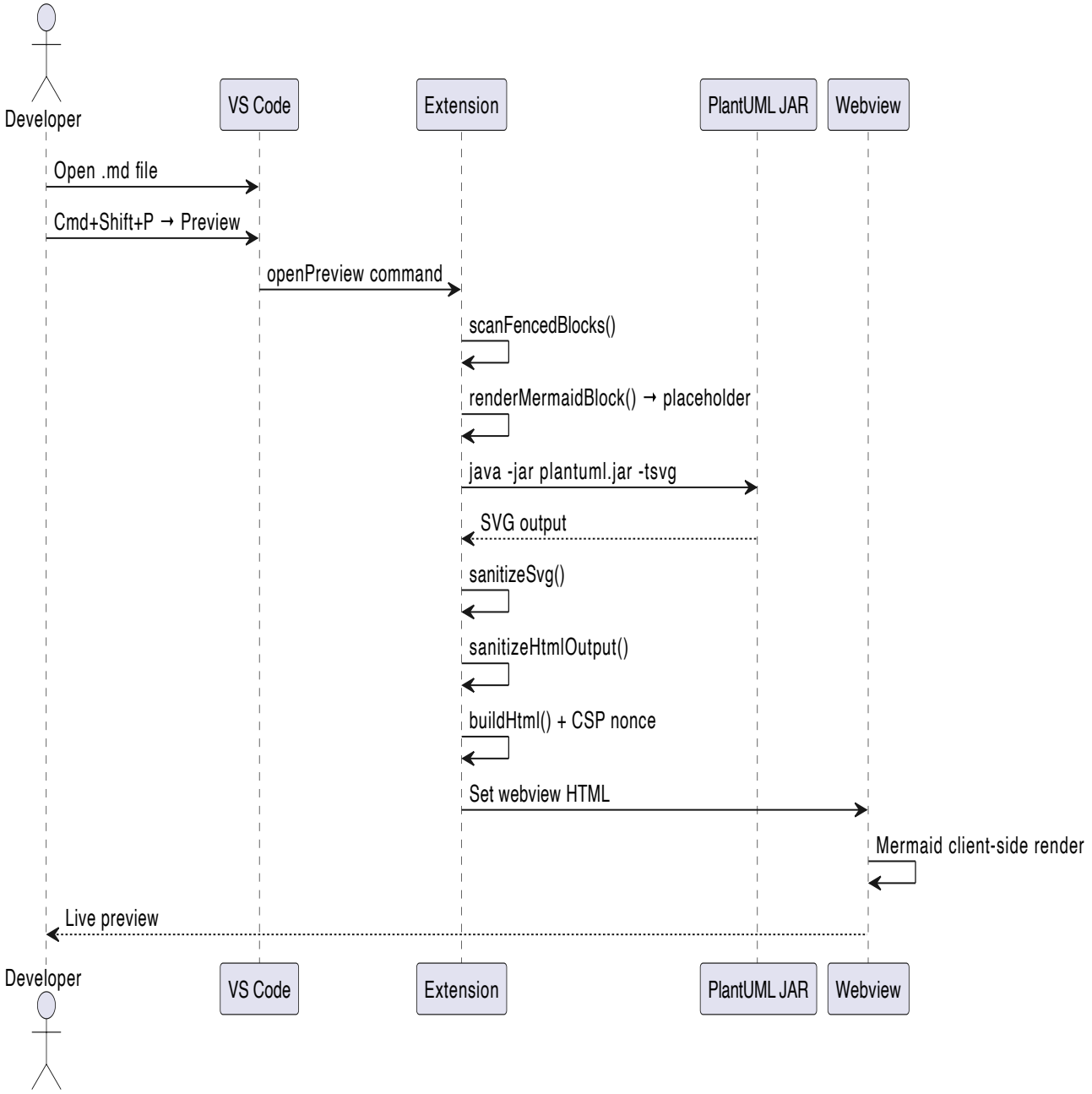
3. PlantUML Diagrams

Rendered locally via bundled JAR. No remote server.

Extension Component Diagram



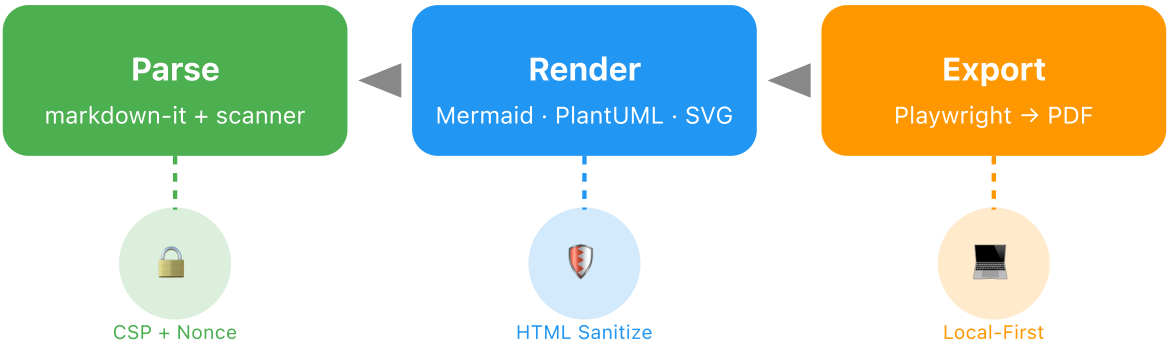
Document Processing Sequence



---

4. Inline SVG (Sanitized)

Dangerous elements (`<script>` , `<foreignObject>` , event handlers) are stripped automatically.



5. Syntax Highlighting

```
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext): void {
  const depManager = new DependencyManager();
  const status = await depManager.ensureAll(context);

  context.subscriptions.push(
    vscode.commands.registerCommand('markdownStudio.openPreview', async () => {
      await openPreviewCommand(context);
    }),
    vscode.commands.registerCommand('markdownStudio.exportPdf', async () => {
      await exportPdfCommand(context);
    })
  );
}
```

```
{
  "markdownStudio.plantuml.mode": "bundled-jar",
  "markdownStudio.java.path": "java",
  "markdownStudio.export.pageFormat": "A4",
  "markdownStudio.security.blockExternalLinks": true
}
```

```
# PlantUML rendering is also useful for Python projects
from dataclasses import dataclass

@dataclass
class DiagramConfig:
    mode: str = "bundled-jar"
    java_path: str = "java"
    timeout_ms: int = 15000
```

6. Images and Security

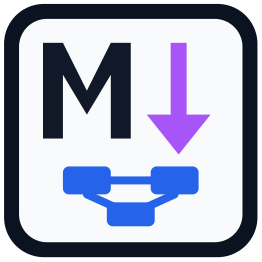
Markdown Studio is local-first. The default security policy (`blockExternalLinks = true`) blocks external resources while allowing local files.

✅ Local image (relative path — works)

- local svg

- icon.png

- icon.svg



Relative paths from the Markdown file are resolved and displayed correctly.

✅ Local image (absolute path — works)

Images with absolute file paths also work in the preview.

❌ External image (blocked by policy)

External image blocked by policy.

External URLs (`https://...`) are blocked by default. The image above shows a policy notice instead.

❌ External link (blocked by policy)

External link — blocked by default ✋

External links are replaced with a blocked notice in the preview.

How to allow external resources

Set `markdownStudio.security.blockExternalLinks` to `false` in VS Code settings to allow external images and links. This is not recommended for corporate/secure environments.

Security Summary

Resource Type	Default	Configurable
Local images (relative path)	✅ Allowed	—
Local images (absolute path)	✅ Allowed	—
External images (https://)	❌ Blocked	`blockExternalLinks: false`
External links (https://)	❌ Blocked	`blockExternalLinks: false`
Inline SVG	✅ Allowed	—
Mermaid diagrams	✅ Allowed	—
PlantUML diagrams	✅ Allowed	—

---

7. Theme Adaptability

Switch between **light** and **dark** mode in VS Code (`Cmd+K Cmd+T`) to see how the preview adapts:

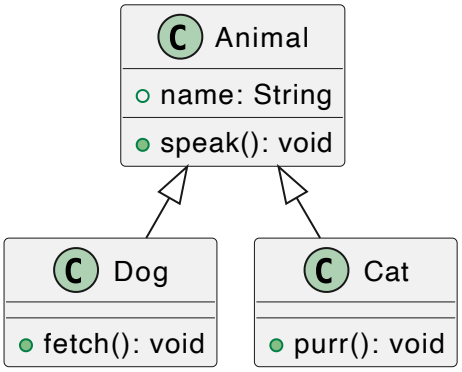
- Mermaid diagrams automatically switch between light and dark themes
- SVG elements use colors chosen for visibility in both themes
- Code blocks use theme-aware syntax highlighting
- PlantUML output receives CSS overrides for dark mode

---

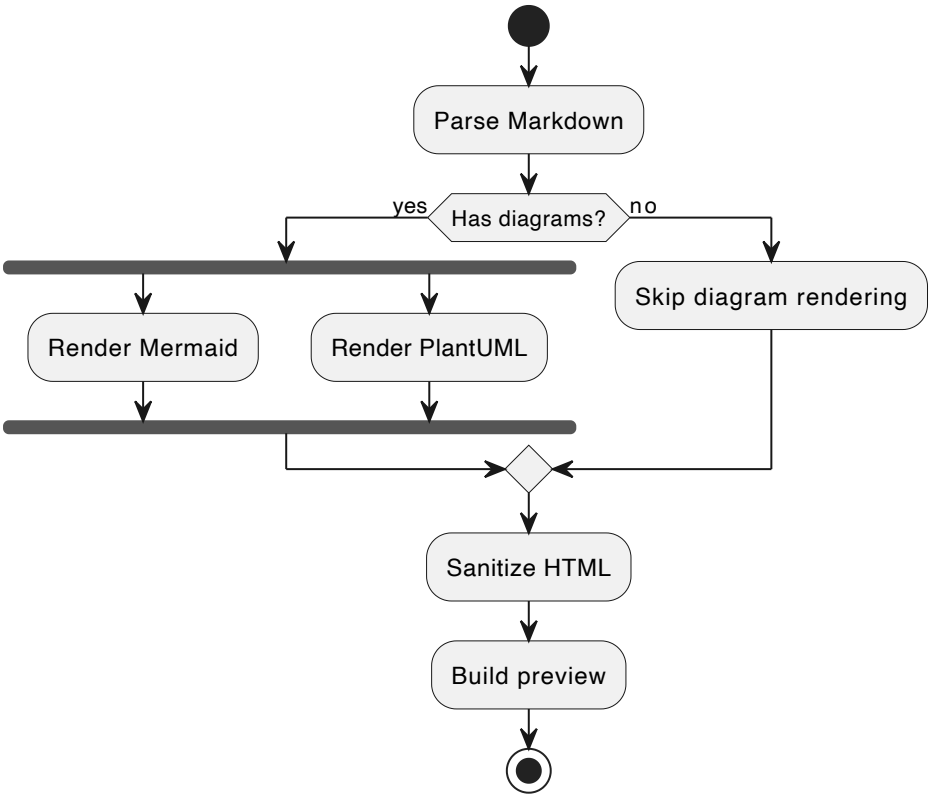
8. Diagram Type Catalog

All diagram types below are verified to render correctly with the bundled PlantUML + Smetana engine (no Graphviz required).

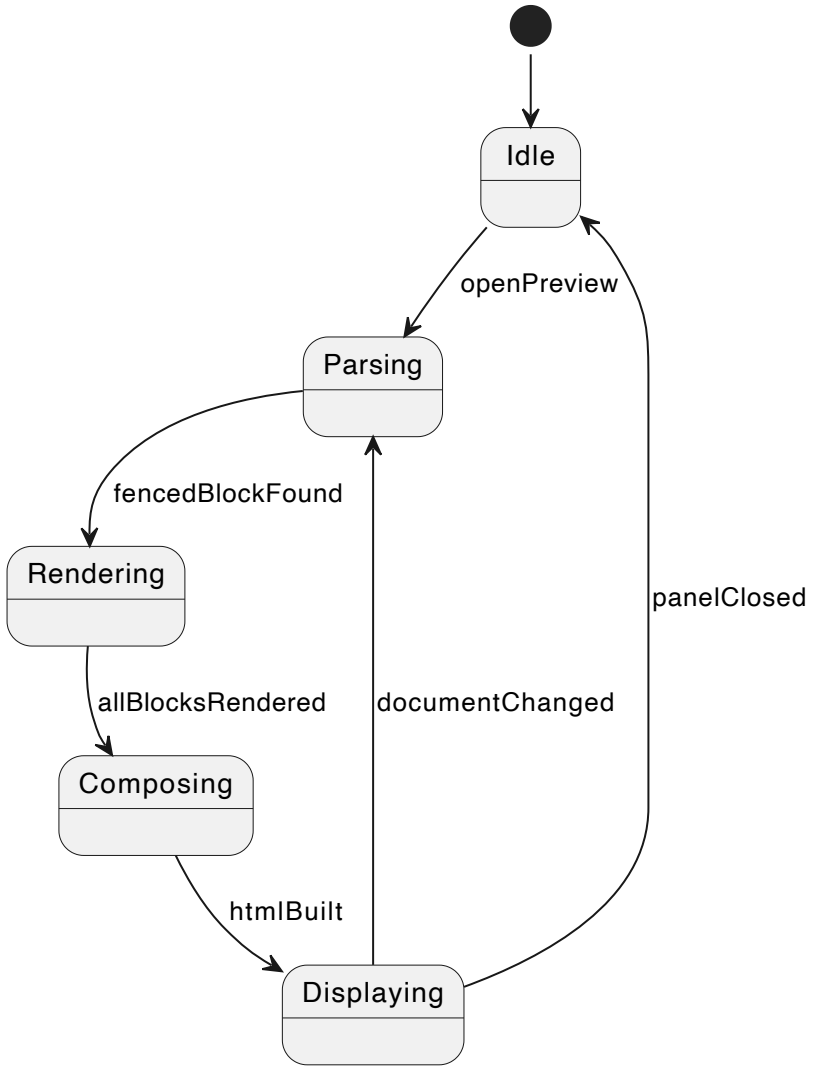
▼ Class Diagram



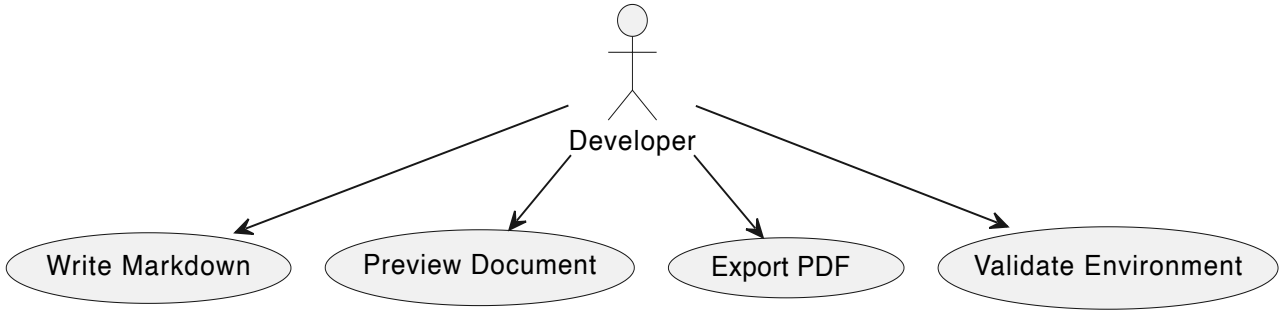
▼ Activity Diagram



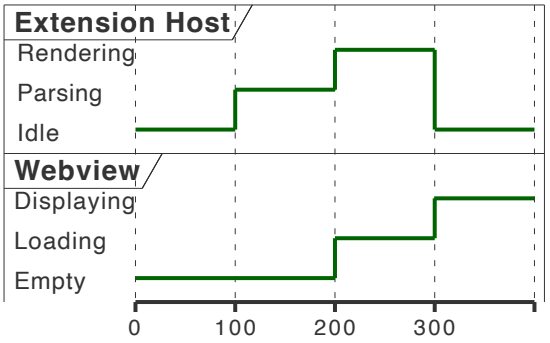
▼ State Diagram



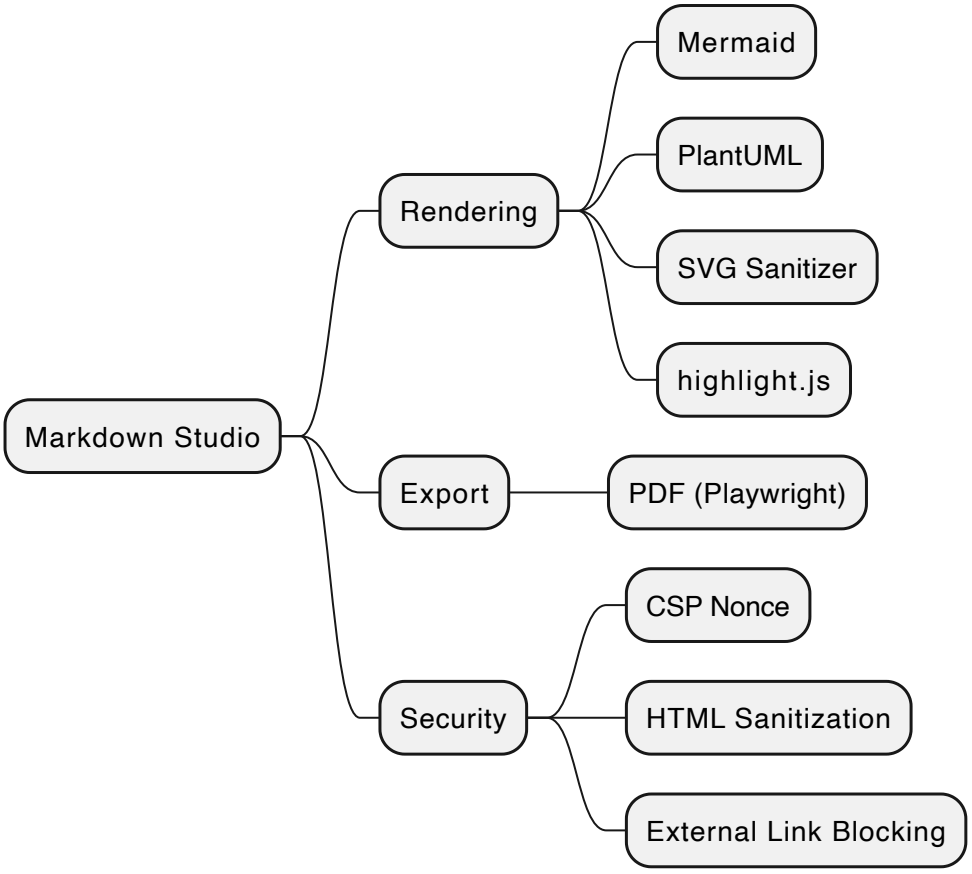
▼ Use Case Diagram



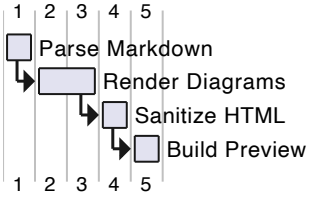
▼ Timing Diagram



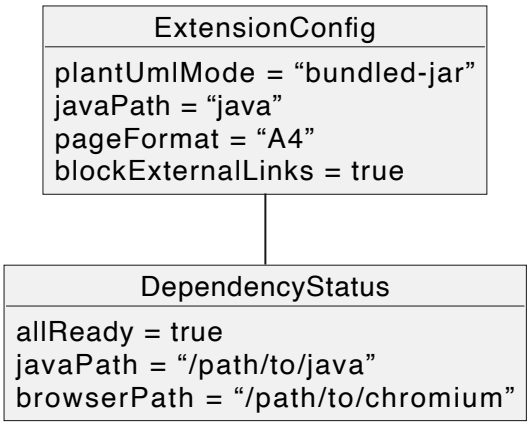
▼ Mind Map



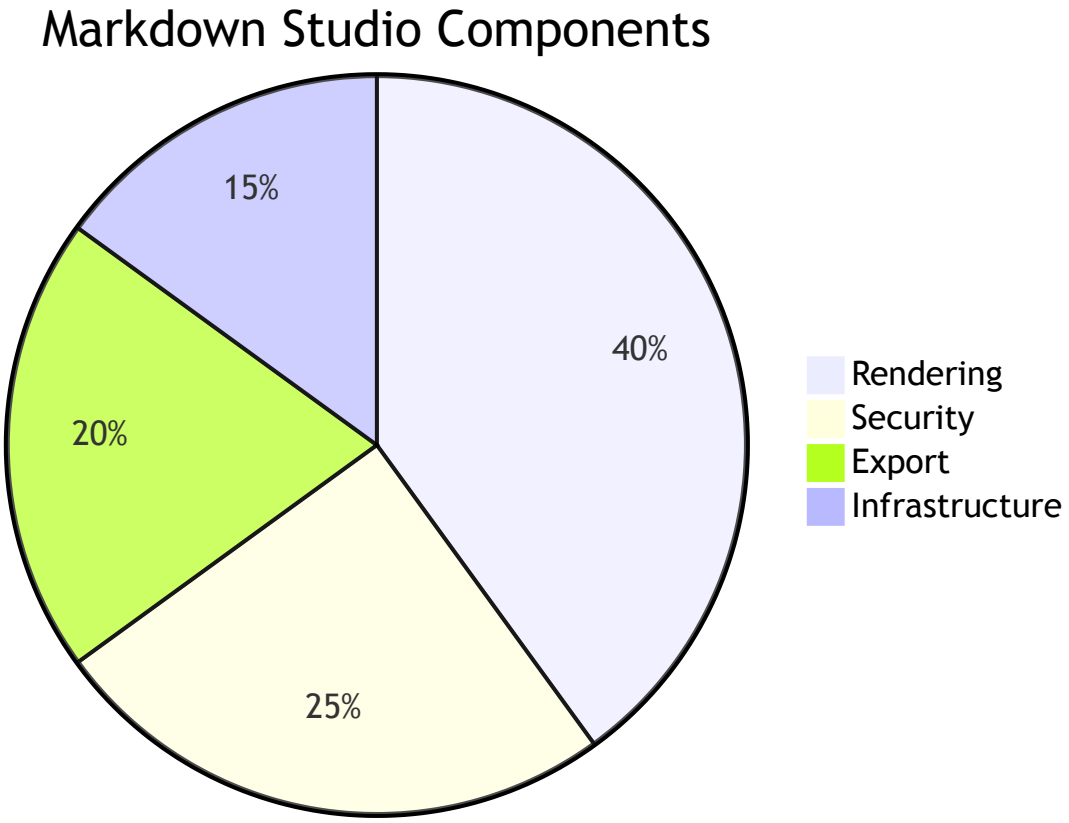
▼ Gantt Chart



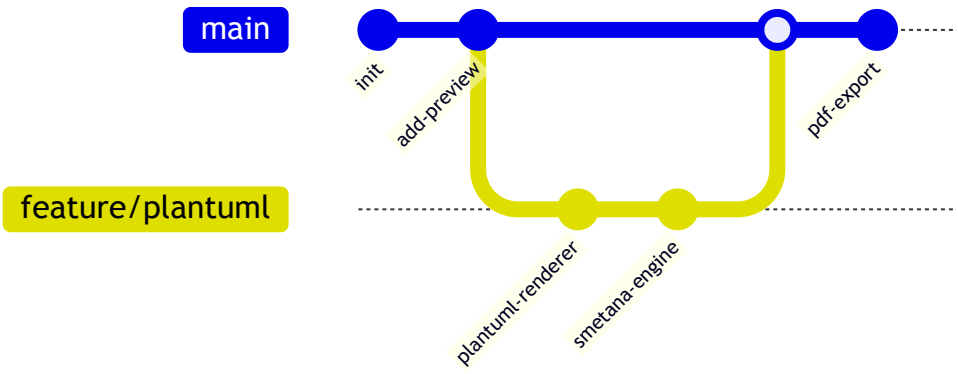
▼ Object Diagram



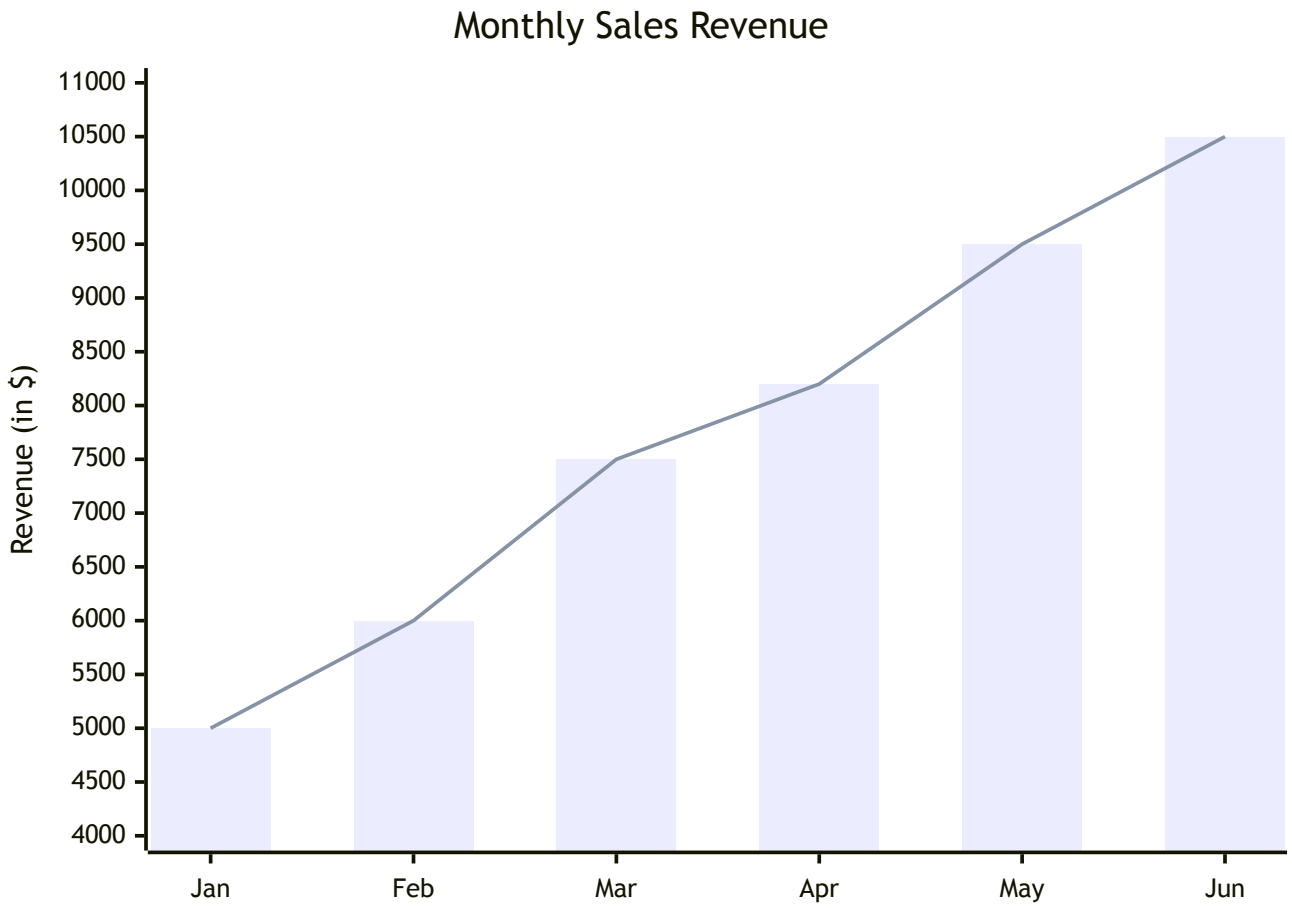
▼ Mermaid: Pie Chart



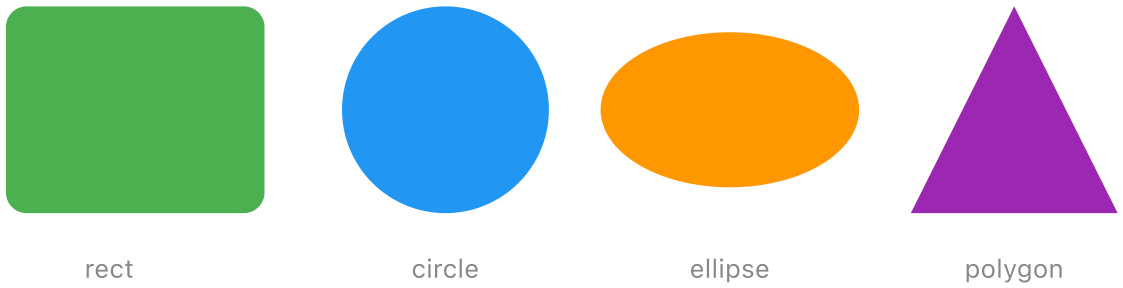
▼ Mermaid: Git Graph



▼ Mermaid: XY Chart

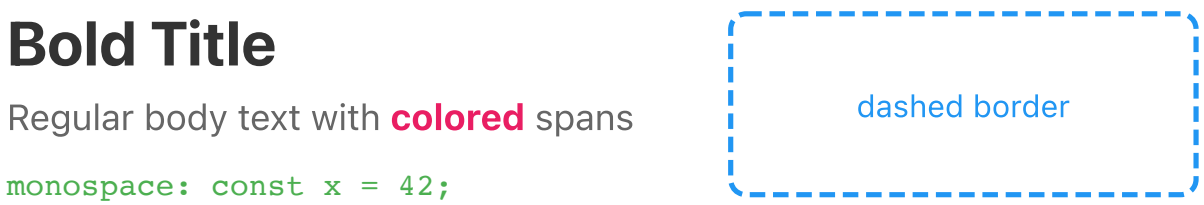


▼ SVG: Basic Shapes

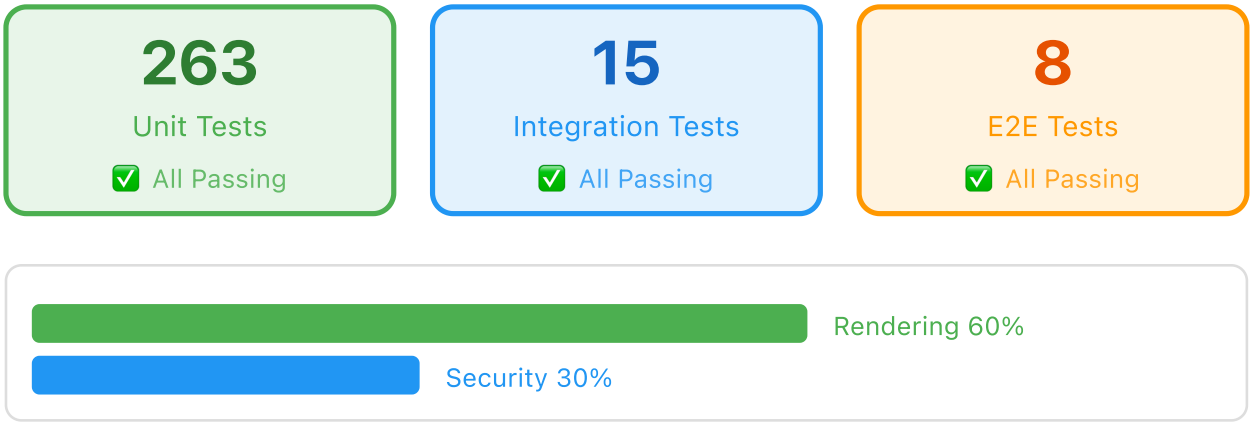


▼ SVG: Lines and Paths

▼ SVG: Text and Styling



▼ SVG: Dashboard Layout



---

9. Custom CSS

Markdown Studio supports built-in themes and inline CSS customization.

Bundled Themes

Select a theme from the settings dropdown (`markdownStudio.style.theme`):

Theme	Description
`default`	No extra styling — uses the selected preset only
`modern`	Indigo accents, soft shadows, refined typography
`markdown-pdf`	Classic look matching the popular Markdown PDF extension
`minimal`	Bare-bones, clean starting point

All themes include dark mode support and print-optimized styles.

Setup

```
// Pick a theme from the dropdown
"markdownStudio.style.theme": "modern"

// Optionally add your own CSS on top
"markdownStudio.style.customCss": "h1 { color: navy; } blockquote { border-left: 3px solid orange; }"
```

How It Works

- Theme CSS is injected **after** the preset styles, so theme rules take priority
- `customCss` is applied **after** the theme, so your inline rules override everything
- The same CSS applies to both preview and PDF export for consistent output
- Copy any bundled theme and customize it to create your own

---

10. PDF Export

This entire document can be exported to PDF:

1. Open this file in VS Code
2. `Cmd+Shift+P` → **Markdown Studio: Export PDF**
3. A `demo.pdf` will be generated next to this file

The PDF uses the same HTML pipeline as the preview.

Markdown Studio v0.2.0